

Day 3: DOM Manipulation & Hotel UI Development

Objectives

By the end of this 4-hour session, you should be able to:

- Set up **Tailwind CSS v4** with the CLI (no Vite yet — that comes Week 2)
 - Style HTML with Tailwind **utility classes**
 - Use the DOM API to render hotel cards from JavaScript data
 - Handle clicks and form input with **event listeners**
 - Explain how today's code maps to **React components** later
-

How to follow this lesson

Rule: After every step → **save** → **refresh the browser** → check the checkpoint.

You need **two terminals** in your project folder:

Terminal	Command	Purpose
1	<code>npm run dev:css</code>	Rebuilds CSS when you add Tailwind classes (leave running)
2	(optional)	Run other commands

Open `index.html` with **Live Server** or double-click it.

Stuck? Full working files are in the lab:

`lab_new/steps/step03_dom_hotel_ui/starter/`

Where this fits

Day	Focus
1–2	Hotel data (JS + TypeScript types)
3 (today)	HTML layout + Tailwind + DOM
4	<code>fetch()</code> for hotel data
5	ES modules + <code>localStorage</code>
6	React + Vite (same UI, new structure)

Tailwind in 30 seconds (read before coding)

Normal CSS: you write rules in a `.css` file.

Tailwind: you add **class names** directly on HTML:

Class	Effect
<code>text-xl</code>	Large text
<code>font-bold</code>	Bold text
<code>bg-white</code>	White background
<code>p-4</code>	Padding on all sides
<code>rounded-lg</code>	Rounded corners
<code>flex</code>	Flexbox layout
<code>grid</code>	Grid layout
<code>hidden</code>	Hide element

We compile classes from `input.css` → `styles.css` using the Tailwind CLI.

Part 1 — Project setup (20 min)

Step 1: Create the folder

```
mkdir hotel-booking-ui
cd hotel-booking-ui
npm init -y
npm install -D tailwindcss @tailwindcss/cli
```

Checkpoint: You have a `hotel-booking-ui` folder with `package.json`.

Step 2: Add build scripts

Open `package.json`. Add a `"scripts"` section:

```
{
  "scripts": {
    "dev:css": "npx @tailwindcss/cli -i ./input.css -o ./styles.css --watch",
    "build:css": "npx @tailwindcss/cli -i ./input.css -o ./styles.css"
  }
}
```

Run once:

```
npm run build:css
```

(It may warn that `input.css` is missing — we create it next.)

Step 3: Create `input.css`

Create a file named `input.css`:

```
@import "tailwindcss";
```

Run:

```
npm run dev:css
```

Leave this terminal running.

Step 4: Create minimal `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Hotel Booking</title>
  <link href="styles.css" rel="stylesheet">
</head>
<body>
  <div id="app">
    <h1>Hello</h1>
  </div>
</body>
</html>
```

Checkpoint: Open `index.html` in the browser. You see plain text **Hello**.

Step 5: Your first Tailwind classes

Change the `<h1>` line to:

```
<h1 class="text-4xl font-bold text-blue-600">Hotel Booking System</h1>
```

Save → refresh.

You should see: A large, bold, blue heading.

What the classes mean:

- `text-4xl` — extra-large text
- `font-bold` — bold weight

- `text-blue-600` — blue colour (built-in Tailwind colour)

https://www.youtube.com/shorts/GO_sp-WI00M?feature=share

Step 6: Page background and spacing

Wrap the heading in a container `div`:

```
<div id="app">
  <div class="min-h-screen bg-gray-50 p-6">
    <h1 class="text-4xl font-bold text-blue-600">Hotel Booking System</h1>
  </div>
</div>
```

Save → refresh.

You should see: Light gray page background with padding around the title.

Class	Meaning
<code>min-h-screen</code>	At least full viewport height
<code>bg-gray-50</code>	Very light gray background
<code>p-6</code>	Padding (1.5rem) on all sides

Step 7: Custom hotel colours (@theme)

Add hotel brand colours to `input.css`:

```
@import "tailwindcss";

@theme {
  --color-hotel-primary: #2c3e50;
  --color-hotel-secondary: #3498db;
  --color-hotel-gold: #f39c12;
}
```

Change the heading class from `text-blue-600` to `text-hotel-primary` and add centering:

```
<h1 class="text-4xl font-bold text-center text-hotel-primary mb-8">
  Hotel Booking System
</h1>
```

Save → refresh. (Tailwind watch should rebuild CSS automatically.)

You should see: Dark blue-gray centered title with space below (`mb-8` = margin-bottom).

Part 2 — Build the layout in HTML

We add the UI **piece by piece**. Only copy the new lines each step.

Step 8: Search box container (white card)

Add **below** the `<h1>`, still inside the gray wrapper:

```
<div class="max-w-4xl mx-auto bg-white rounded-lg shadow-md p-6 mb-8">
  <p class="text-gray-500">Search area – inputs coming next</p>
</div>
```

Save → refresh.

You should see: A white card with shadow, centered, max width.

Class	Meaning
<code>max-w-4xl</code>	Don't grow wider than ~896px
<code>mx-auto</code>	Center horizontally
<code>bg-white</code>	White background
<code>rounded-lg</code>	Rounded corners

Class	Meaning
<code>shadow-md</code>	Medium drop shadow

Step 9: Search text input

Replace the placeholder `<p>` inside the white card with:

```
<input
  type="text"
  id="searchInput"
  placeholder="Search hotels..."
  class="w-full border border-gray-300 rounded-lg px-4 py-2"
>
```

Save → refresh.

You should see: One styled text input.

Class	Meaning
<code>w-full</code>	Full width of parent
<code>border border-gray-300</code>	Light gray border
<code>px-4 py-2</code>	Horizontal + vertical padding inside input

Step 10: Location dropdown and max price

Replace the single `<input>` with a **grid** of three controls:

```
<div class="grid grid-cols-1 md:grid-cols-3 gap-4">
  <input
    type="text"
    id="searchInput"
    placeholder="Search hotels..."
    class="border border-gray-300 rounded-lg px-4 py-2 focus:outline-none
focus:border-hotel-secondary"
  >
```

```

<select
  id="locationFilter"
  class="border border-gray-300 rounded-lg px-4 py-2 focus:outline-none
focus:border-hotel-secondary"
>
  <option value="">All Locations</option>
  <option value="Foshan">Foshan</option>
  <option value="Guangzhou">Guangzhou</option>
  <option value="Shenzhen">Shenzhen</option>
</select>
<input
  type="number"
  id="maxPrice"
  placeholder="Max Price"
  class="border border-gray-300 rounded-lg px-4 py-2 focus:outline-none
focus:border-hotel-secondary"
>
</div>

```

Save → refresh.

You should see: Three fields in a row (on wide screen) or stacked (on narrow screen — resize the window to test).

Class	Meaning
<code>grid grid-cols-1</code>	1 column on mobile
<code>md:grid-cols-3</code>	3 columns from medium screens up
<code>gap-4</code>	Space between grid items
<code>focus:border-hotel-secondary</code>	Blue border when input is focused

Step 11: Empty hotel list area

Add **below** the white search card:

```

<div
  id="hotelList"
  class="max-w-4xl mx-auto grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3
gap-6"
>
  <p class="text-gray-400 col-span-full text-center">Hotels will appear
here</p>
</div>

```


Save → refresh.

You should see: Placeholder text where cards will go.

We will remove that `<p>` with JavaScript in Step 14.

Class	Meaning
col-span-1	takes 1 column
col-span-2	takes 2 columns
col-span-full	takes all available columns

Step 12: Booking modal (hidden for now)

Add **below** `#hotelList`, still inside the gray wrapper:

```
<div
  id="bookingModal"
  class="hidden fixed inset-0 bg-black/50 flex items-center justify-center
z-50"
>
  <div class="bg-white rounded-lg p-8 max-w-md w-full mx-4">
    <h2 class="text-2xl font-bold text-hotel-primary mb-4">Book Your
Stay</h2>
    <p class="text-gray-500">Form coming in Step 20</p>
  </div>
</div>
```

Save → refresh.

You should see: Nothing new — `hidden` keeps the modal invisible. That is correct.

Class	Meaning
hidden	display: none
fixed inset-0	Cover full screen
bg-black/50	Semi-transparent black overlay
flex items-center justify-center	Center the white box

Class

z-50

Meaning

Make sure modal is above others

Step 13: Link `app.js`

Add before `</body>` in `index.html` – at the bottom:

```
<script src="app.js"></script>
```

Create `app.js` with one line:

```
console.log('app.js loaded');
```

Save → refresh → open browser DevTools (F12) → Console.

You should see: `app.js loaded`

Part 3 — JavaScript + DOM

Step 14: Hotel data array

Replace `app.js` with:

```
const hotels = [
  {
    id: 1,
    name: 'Grand Palace Hotel',
    location: 'Foshan',
    price: 200,
    rating: 4.5,
    available: true,
    amenities: ['WiFi', 'Pool', 'Spa'],
  },
  {
    id: 2,
    name: 'Seaside Resort',
```

```
    location: 'Guangzhou',
    price: 150,
    rating: 4.0,
    available: true,
    amenities: ['WiFi', 'Beach', 'Pool'],
  },
  {
    id: 3,
    name: 'Mountain Lodge',
    location: 'Shenzhen',
    price: 180,
    rating: 4.8,
    available: false,
    amenities: ['WiFi', 'Fireplace', 'Hiking'],
  },
];

console.log(hotels);
```

Checkpoint: Console shows an array of 3 hotels. Page still shows placeholder text.

Step 15: Display one hotel card (simplest version)

Add to `app.js`:

```
function displayHotels(hotelsToShow) {
  const hotelList = document.getElementById('hotelList');

  hotelList.innerHTML = ''; // Clear old hotels

  hotelsToShow.forEach((hotel) => {
    const card = document.createElement('div');
    card.className = 'bg-white rounded-lg shadow-md p-6';
    card.textContent = hotel.name;
    hotelList.appendChild(card);
  });
}

displayHotels(hotels);
```

Save → refresh.

You should see: Three plain white boxes with hotel names only.

DOM recap:

- `getElementById` — find `#hotelList`
 - `createElement` — make a new `<div>`
 - `className` — set Tailwind classes
 - `appendChild` — add to the page
-

Step 16: Richer card with `innerHTML`

Replace the `forEach` body inside `displayHotels` with a `createHotelCard` function:

```
function createHotelCard(hotel) {
  const card = document.createElement('article');
  card.className = 'bg-white rounded-lg shadow-md p-6';

  card.innerHTML = `
    <h3 class="text-xl font-bold text-hotel-primary mb-2">${hotel.name}</h3>
    <p class="text-gray-600 mb-2">${hotel.location}</p>
    <p class="text-2xl font-bold text-hotel-
secondary">${hotel.price}/night</p>
  `;

  return card;
}

function displayHotels(hotelsToShow) {
  const hotelList = document.getElementById('hotelList');
  hotelList.innerHTML = '';

  hotelsToShow.forEach((hotel) => {
    hotelList.appendChild(createHotelCard(hotel));
  });
}

displayHotels(hotels);
```

Save → refresh.

You should see: Cards with name, location, and price styled with Tailwind.

Step 17: Rating, amenities, availability

Update `createHotelCard innerHTML` to:

```
const availabilityClass = hotel.available ? 'text-green-600' : 'text-red-600';
const availabilityText = hotel.available ? 'Available' : 'Booked';

card.innerHTML = `
  <h3 class="text-xl font-bold text-hotel-primary mb-2">${hotel.name}</h3>
  <p class="text-gray-600 mb-2">📍 ${hotel.location}</p>
  <p class="text-2xl font-bold text-hotel-secondary mb-2">${hotel.price}/night</p>
  <p class="text-hotel-gold mb-3">★ ${hotel.rating} stars</p>
  <div class="flex flex-wrap gap-2 mb-3">
    ${hotel.amenities
      .map((a) => `<span class="bg-gray-100 px-2 py-1 rounded text-sm">${a}</span>`)
      .join('')}
  </div>
  <p class="${availabilityClass}">${availabilityText}</p>
`;
```

Save → refresh.

You should see: Stars, amenity tags, green "Available" or red "Booked".

Class	Meaning
<code>flex flex-wrap gap-2</code>	Tags flow in a row and wrap
<code>text-sm</code>	Smaller text for tags

Step 18: "Book Now" button

Inside `createHotelCard`, after the availability `<p>`, add a button **only if available**:

```
card.innerHTML = `
  <h3 class="text-xl font-bold text-hotel-primary mb-2">${hotel.name}</h3>
  <p class="text-gray-600 mb-2">📍 ${hotel.location}</p>
  <p class="text-2xl font-bold text-hotel-secondary mb-2">${hotel.price}/night</p>
  <p class="text-hotel-gold mb-3">★ ${hotel.rating} stars</p>
  <div class="flex flex-wrap gap-2 mb-3">
    ${hotel.amenities
      .map((a) => `<span class="bg-gray-100 px-2 py-1 rounded text-sm">${a}</span>`)
      .join('')}
  </div>
  <p class="${availabilityClass}">${availabilityText}</p>
  <button class="text-hotel-gold font-weight-bold mb-3" ${hotel.available ? '' : 'hidden'}>Book Now</button>
`;
```

```

</div>
<p class="${availabilityClass} mb-4">${availabilityText}</p>
${
  hotel.available
    ? `<button
      type="button"
      data-book-hotel="${hotel.id}"
      class="w-full bg-hotel-secondary text-white py-2 rounded
hover:bg-hotel-primary"
      >
        Book Now
      </button>`
    : ''
  }
`;

```

Save → refresh.

`data-book-hotel` is a **custom HTML data attribute**. It's used to **store extra information** (in this case, the hotel's ID) on the button.

You should see: Blue "Book Now" on available hotels; Mountain Lodge has no button.

Class	Meaning
<code>w-full</code>	Button spans card width
<code>bg-hotel-secondary text-white</code>	Blue button, white text
<code>hover:bg-hotel-primary</code>	Darker on mouse hover

Part 4 — Events: search and filter

Step 19: Filter when user types

Add to `app.js`:

```

function filterHotels() {
  const searchTerm =
document.getElementById('searchInput').value.toLowerCase();
  const locationFilter = document.getElementById('locationFilter').value;
  const maxPrice = document.getElementById('maxPrice').value;

  const filtered = hotels.filter((hotel) => {
    const matchesSearch =

```

```

    hotel.name.toLowerCase().includes(searchTerm) ||
    hotel.location.toLowerCase().includes(searchTerm);
    const matchesLocation = !locationFilter || hotel.location ===
locationFilter;
    const matchesPrice = !maxPrice || hotel.price <= Number(maxPrice);
    return matchesSearch && matchesLocation && matchesPrice;
  });

  displayHotels(filtered);
}

document.getElementById('searchInput').addEventListener('input',
filterHotels);
document.getElementById('locationFilter').addEventListener('change',
filterHotels);
document.getElementById('maxPrice').addEventListener('input', filterHotels);

```

Save → refresh.

Test:

1. Type **Grand** in search → 1 card
2. Select **Guangzhou** → Seaside Resort only
3. Set max price **160** → cheaper hotels only

React preview: Later you'll store search text in **useState** instead of reading the DOM.

Part 5 — Booking modal

Step 20: Replace modal placeholder with a form

In **index.html**, replace the inside of **#bookingModal** (keep the outer overlay **div**) with:

```

<div class="bg-white rounded-lg p-6 md:p-8 max-w-md w-full mx-4">
  <h2 class="text-2xl font-bold text-hotel-primary mb-6">Book Your Stay</h2>

  <form id="bookingForm" class="space-y-4">
    <div>
      <label class="block text-gray-700 mb-2" for="guestName">Guest

```

```

Name</label>
  <input type="text" id="guestName" required
    class="w-full border border-gray-300 rounded-lg px-4 py-2">
</div>
<div>
  <label class="block text-gray-700 mb-2" for="email">Email</label>
  <input type="email" id="email" required
    class="w-full border border-gray-300 rounded-lg px-4 py-2">
</div>
<div>
  <label class="block text-gray-700 mb-2" for="checkIn">Check-in</label>
  <input type="date" id="checkIn" required
    class="w-full border border-gray-300 rounded-lg px-4 py-2">
</div>
<div>
  <label class="block text-gray-700 mb-2" for="checkOut">Check-
out</label>
  <input type="date" id="checkOut" required
    class="w-full border border-gray-300 rounded-lg px-4 py-2">
</div>
<div class="bg-gray-50 rounded-lg p-4">
  <p class="text-lg font-bold">Total: <span id="totalPrice">$0</span>
</p>
  <p>Nights: <span id="totalNights">0</span></p>
</div>
<div class="flex gap-4">
  <button type="submit"
    class="flex-1 bg-hotel-secondary text-white py-2 rounded hover:bg-
hotel-primary">
    Confirm Booking
  </button>
  <button type="button" id="closeModal"
    class="flex-1 bg-gray-300 text-gray-700 py-2 rounded hover:bg-gray-
400">
    Cancel
  </button>
</div>
</form>
</div>

```

Checkpoint: Modal still hidden — no visible change yet. Good.

Step 21: Open and close the modal

Add to `app.js`:

```

let selectedHotel = null;

function openBookingModal(hotelId) {

```



```

selectedHotel = hotels.find((h) => h.id === hotelId) ?? null;
document.getElementById('bookingModal').classList.remove('hidden');
}

function closeBookingModal() {
  document.getElementById('bookingModal').classList.add('hidden');
  selectedHotel = null;
}

document.getElementById('hotelList').addEventListener('click', (event) => {
  const button = event.target.closest('[data-book-hotel]');
  if (!button) return;
  openBookingModal(Number(button.dataset.bookHotel));
});

document.getElementById('closeModal').addEventListener('click',
closeBookingModal);

```

Save → refresh.

Test: Click **Book Now** → modal appears. Click **Cancel** → modal hides.

Why `closest`? Buttons are created dynamically. One listener on `#hotelList` handles all "Book Now" clicks.

Step 22: Calculate total price from dates

Add to `app.js`:

```

function calculateTotal() {
  const checkIn = document.getElementById('checkIn').value;
  const checkOut = document.getElementById('checkOut').value;
  if (!checkIn || !checkOut || !selectedHotel) return;

  const nights = Math.ceil(
    (new Date(checkOut) - new Date(checkIn)) / (1000 * 60 * 60 * 24)
  );

  if (nights > 0) {
    document.getElementById('totalNights').textContent = String(nights);
    document.getElementById('totalPrice').textContent = `$$${nights *
selectedHotel.price}`;
  }
}

document.getElementById('checkIn').addEventListener('change',
calculateTotal);

```

```
document.getElementById('checkOut').addEventListener('change',  
calculateTotal);
```

Test: Open modal → pick check-in and check-out → nights and total update.

Step 23: Submit the booking

Add to `app.js`:

```
document.getElementById('bookingForm').addEventListener('submit', (event) =>  
{  
  event.preventDefault();  
  if (!selectedHotel) return;  
  
  const booking = {  
    hotelName: selectedHotel.name,  
    guestName: document.getElementById('guestName').value,  
    email: document.getElementById('email').value,  
    checkIn: document.getElementById('checkIn').value,  
    checkOut: document.getElementById('checkOut').value,  
    total: document.getElementById('totalPrice').textContent,  
  };  
  
  console.log('Booking:', booking);  
  alert(`Booking confirmed for ${booking.hotelName}!`);  
  closeBookingModal();  
});
```

Test: Fill the form → Confirm → alert appears, modal closes.

Part 6 — Polish

Step 24: Card hover and fade-in animation

Add to `input.css` inside `@theme`:

```
--animate-fade-in: fade-in 0.5s ease-out;

@keyframes fade-in {
  from { opacity: 0; transform: translateY(20px); }
  to   { opacity: 1; transform: translateY(0); }
}
```

Update `createHotelCard` first line of `className`:

```
card.className =
  'bg-white rounded-lg shadow-md p-6 hover:shadow-xl hover:-translate-y-1
  transition-all duration-300 animate-fade-in';
```

Save → refresh.

You should see: Cards fade in; they lift slightly on hover.

Step 25: Test responsive layout

Resize the browser window:

Width	What to check
Narrow (phone)	Search fields stack vertically; 1 hotel column
Medium (tablet)	2 hotel columns
Wide (desktop)	3 search fields in a row; 3 hotel columns

No code change — just verify the `md:` and `lg:` classes from earlier steps.

Bridge to React (class discussion)

You already built these "components" without React:

```
App
├─ SearchBar    → #searchInput, #locationFilter, #maxPrice
├─ HotelList    → #hotelList
```

```
|   | HotelCard    → createHotelCard(hotel)
|   | BookingModal → #bookingModal
```

On **Day 6** the same Tailwind classes move into JSX:

```
<HotelCard name={hotel.name} price={hotel.price} onBook={() =>
  openBooking(hotel.id)} />
```

Troubleshooting

Problem	Fix
No styles / plain HTML	Run <code>npm run build:css</code> from inside <code>hotel-booking-ui</code>
<code>ENOENT</code> <code>package.json</code>	You are in the wrong folder — <code>cd hotel-booking-ui</code> first
JavaScript text on page	Broken HTML — check every tag has closing <code>"</code> before <code>></code>
Custom colours don't work	Check <code>input.css @theme</code> block; ensure <code>dev:css</code> is running
Cards don't appear	Open DevTools Console for errors in <code>app.js</code>

Homework

Pick **2-3**:

1. Show "No hotels found" when filter returns empty array
2. Add form validation (check-out must be after check-in)
3. Add hotel images with `rounded-lg`
4. Add ★ favorites button (persist on Day 5 with `localStorage`)

Summary

Step	You learned
1–7	Tailwind setup, first classes, <code>@theme</code> colours
8–12	HTML layout with utility classes
13–18	DOM: <code>createElement</code> , <code>innerHTML</code> , dynamic cards
19	Event listeners + <code>filter</code>
20–23	Modal, form, price calculation
24–25	Animation + responsive design

Next (Day 4): Keep this project. Replace `const hotels = [...]` with `fetch()` — rendering code stays the same.

Lab reference: lab_new/steps/step03_dom_hotel_ui/starter/

Appendix: Complete working project

After Step 25, your `hotel-booking-ui/` folder should look like this:

```
hotel-booking-ui/  
├── index.html      # Page layout  
├── input.css       # Tailwind source + theme  
├── styles.css      # Generated (run npm run build:css)  
├── app.js          # DOM logic  
├── package.json  
└── node_modules/  # After npm install
```

Run from inside `hotel-booking-ui`:

```
npm install  
npm run build:css
```

Open `index.html` in the browser. For development, keep `npm run dev:css` running in a terminal.

package.json

```
{
  "name": "hotel-booking-ui",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev:css": "npx @tailwindcss/cli -i ./input.css -o ./styles.css --watch",
    "build:css": "npx @tailwindcss/cli -i ./input.css -o ./styles.css"
  },
  "devDependencies": {
    "@tailwindcss/cli": "^4.1.11",
    "tailwindcss": "^4.1.11"
  }
}
```

input.css

```
@import "tailwindcss";

@theme {
  --color-hotel-primary: #2c3e50;
  --color-hotel-secondary: #3498db;
  --color-hotel-accent: #e74c3c;
  --color-hotel-gold: #f39c12;

  --animate-fade-in: fade-in 0.5s ease-out;

  @keyframes fade-in {
    from {
      opacity: 0;
      transform: translateY(20px);
    }
    to {
      opacity: 1;
      transform: translateY(0);
    }
  }
}
```

index.html

```

    <h2 class="text-2xl font-bold text-hotel-primary mb-6">Book Your
    Stay</h2>

    <form id="bookingForm" class="space-y-4">
      <div>
        <label class="block text-gray-700 mb-2" for="guestName">Guest
        Name</label>
        <input
          type="text"
          id="guestName"
          required
          class="w-full border border-gray-300 rounded-lg px-4 py-2
          focus:outline-none focus:border-hotel-secondary"
        >
      </div>

      <div>
        <label class="block text-gray-700 mb-2"
        for="email">Email</label>
        <input
          type="email"
          id="email"
          required
          class="w-full border border-gray-300 rounded-lg px-4 py-2
          focus:outline-none focus:border-hotel-secondary"
        >
      </div>

      <div>
        <label class="block text-gray-700 mb-2" for="checkIn">Check-in
        Date</label>
        <input
          type="date"
          id="checkIn"
          required
          class="w-full border border-gray-300 rounded-lg px-4 py-2
          focus:outline-none focus:border-hotel-secondary"
        >
      </div>

      <div>
        <label class="block text-gray-700 mb-2" for="checkOut">Check-
        out Date</label>
        <input
          type="date"
          id="checkOut"
          required
          class="w-full border border-gray-300 rounded-lg px-4 py-2
          focus:outline-none focus:border-hotel-secondary"
        >
      </div>

      <div class="bg-gray-50 rounded-lg p-4">
        <p class="text-lg font-bold">
          Total: <span id="totalPrice">$0</span>
        </p>
        <p>Nights: <span id="totalNights">0</span></p>
      </div>
    </form>
  
```



```
    </div>

    <div class="flex gap-4">
      <button
        type="submit"
        class="flex-1 bg-hotel-secondary text-white py-2 rounded
hover:bg-hotel-primary transition-colors"
      >
        Confirm Booking
      </button>
      <button
        type="button"
        id="closeModal"
        class="flex-1 bg-gray-300 text-gray-700 py-2 rounded
hover:bg-gray-400 transition-colors"
      >
        Cancel
      </button>
    </div>
  </form>
</div>
</div>
</div>
<script src="app.js"></script>
</body>
</html>
```

app.js

```
const hotels = [
  {
    id: 1,
    name: 'Grand Palace Hotel',
    location: 'Foshan',
    price: 200,
    rating: 4.5,
    available: true,
    amenities: ['WiFi', 'Pool', 'Spa'],
  },
  {
    id: 2,
    name: 'Seaside Resort',
    location: 'Guangzhou',
    price: 150,
    rating: 4.0,
    available: true,
    amenities: ['WiFi', 'Beach', 'Pool'],
  },
  {
```



```

hotelsToShow.forEach((hotel) => {
  hotelList.appendChild(createHotelCard(hotel));
});
}

function filterHotels() {
  const searchTerm =
document.getElementById('searchInput').value.toLowerCase();
  const locationFilter = document.getElementById('locationFilter').value;
  const maxPrice = document.getElementById('maxPrice').value;

  const filtered = hotels.filter((hotel) => {
    const matchesSearch =
      hotel.name.toLowerCase().includes(searchTerm) ||
      hotel.location.toLowerCase().includes(searchTerm);
    const matchesLocation = !locationFilter || hotel.location ===
locationFilter;
    const matchesPrice = !maxPrice || hotel.price <= Number(maxPrice);

    return matchesSearch && matchesLocation && matchesPrice;
  });

  displayHotels(filtered);
}

function openBookingModal(hotelId) {
  selectedHotel = hotels.find((h) => h.id === hotelId) ?? null;
  document.getElementById('bookingModal').classList.remove('hidden');
}

function closeBookingModal() {
  document.getElementById('bookingModal').classList.add('hidden');
  selectedHotel = null;
  document.getElementById('bookingForm').reset();
  document.getElementById('totalNights').textContent = '0';
  document.getElementById('totalPrice').textContent = '$0';
}

function calculateTotal() {
  const checkIn = document.getElementById('checkIn').value;
  const checkOut = document.getElementById('checkOut').value;

  if (!checkIn || !checkOut || !selectedHotel) return;

  const startDate = new Date(checkIn);
  const endDate = new Date(checkOut);
  const nights = Math.ceil((endDate - startDate) / (1000 * 60 * 60 * 24));

  if (nights > 0) {
    document.getElementById('totalNights').textContent = String(nights);
    document.getElementById('totalPrice').textContent = `$$${nights *
selectedHotel.price}`;
  }
}

// Search and filter

```

```
document.getElementById('searchInput').addEventListener('input',
filterHotels);
document.getElementById('locationFilter').addEventListener('change',
filterHotels);
document.getElementById('maxPrice').addEventListener('input', filterHotels);

// Booking modal
document.getElementById('hotelList').addEventListener('click', (event) => {
  const button = event.target.closest('[data-book-hotel]');
  if (!button) return;
  openBookingModal(Number(button.dataset.bookHotel));
});

document.getElementById('closeModal').addEventListener('click',
closeBookingModal);
document.getElementById('checkIn').addEventListener('change',
calculateTotal);
document.getElementById('checkOut').addEventListener('change',
calculateTotal);

document.getElementById('bookingForm').addEventListener('submit', (event) =>
{
  event.preventDefault();

  if (!selectedHotel) return;

  const booking = {
    hotelId: selectedHotel.id,
    hotelName: selectedHotel.name,
    guestName: document.getElementById('guestName').value,
    email: document.getElementById('email').value,
    checkIn: document.getElementById('checkIn').value,
    checkOut: document.getElementById('checkOut').value,
    total: document.getElementById('totalPrice').textContent,
  };

  console.log('Booking submitted:', booking);
  alert(`Booking confirmed for ${booking.hotelName}!`);
  closeBookingModal();
});

// Initial render
displayHotels(hotels);
```